Министерство образования Республики Беларусь
Учреждение образования
«Брестский Государственный технический университет»
Кафедра ИИТ

Лабораторная работа №4
По дисциплине «АПО ЭВМиС»
Тема: «Изучение аппаратно-программной архитектуры процессоров семейства Intel – программирование сопроцессора».

Выполнил:
Студент 4-го курса
Группы АС-66
Невар В.А.
Проверила:
Кулик А.Д.

Брест 2026

Цель работы: изучить архитектуру и программную модель сопроцессора; получить практические навыки работы с системами целочисленных и вещественных команд сопроцессора.

Вариант №14

Задание: вычислить значение функции $f(x, y) = (1 + x) / (5x^2 - y^2 + 1)$ при $x - y < -4$; $3x + 5y^2$ при $x - y > 6$; $x^3 - 2y$ при $-4 \le x - y \le 6$. В разделе 1 использовать регистры сопроцессора и целочисленные команды, в разделе 2 – вещественные. Каждую ветку алгоритма оформить отдельной процедурой, ввод и вывод выполнять через консоль, программу протестировать по всем условиям. Среда разработки – MS Visual Studio (MASM, платформа x64).

Ход работы

Алгоритм. Оба раздела работают по одной схеме. В регистрах сопроцессора вычисляется разность $x - y$ и командой fcomip сравнивается с числами -4 и 6; по результату выбирается одна из трёх веток, каждая оформлена отдельной процедурой. В ветви 1 знаменатель сравнивается с нулём, при равенстве выводится сообщение о делении на ноль. Значения x и y ограничены диапазоном $[-1000; 1000]$, ввод проверяется на корректность, пустая строка завершает раздел.

Раздел 1 (целочисленные команды). Числа x и y – целые, они загружаются в стек сопроцессора командой fild; вычисления выполняют fiadd, fisub, fimul и fidiv с операндами в памяти, промежуточные результаты складываются и вычитаются командами faddp и fsubp. Результат записывается командой fistp в 32-разрядную ячейку с округлением до ближайшего целого. Процедуры: BranchInt1, BranchInt2, BranchInt3, выбор ветки – CalcInt.

Раздел 2 (вещественные команды). Числа x и y – вещественные (REAL8), строку вида «-12.345» в число переводит сама программа: цифры накапливаются командами fimul и fiadd, затем число делится на 10 по количеству цифр после точки. Вычисления выполняют fld, fadd, fsub, fmul, faddp, fsubp и fdivp, результат записывается командой fstp; при выводе он умножается на 10^6, округляется командой frndint и печатается с шестью знаками после точки. Процедуры: BranchReal1, BranchReal2, BranchReal3, CalcReal.

Проверка. Программа поочерёдно выполняет разделы 1 и 2: запрашивает x и y, выводит номер ветки и значение функции. Проверены все ветки и деление на ноль. Раздел 1: $(-3; 7)$ – 1, $(4; 9)$ – деление на ноль, $(5; -3)$ – 60, $(2; 1)$ – 6. Раздел 2: $(-3; 7)$ – 0.666667, $(4; 9)$ – деление на ноль, $(7.5; 0.5)$ – 23.750000, $(2.5; 1)$ – 13.625000.

Текст программы:

```
; Laboratory work No. 4. Variant 14
; Coprocessor (x87 FPU) programming.
; Function f(x, y):
;     (1 + x) / (5x^2 - y^2 + 1)   if x - y < -4
;     3x + 5y^2                    if x - y > 6
;     x^3 - 2y                     if -4 <= x - y <= 6
```

```
; Part 1 uses the integer FPU instructions (FILD, FIADD, FIMUL, FISUB, FISTP),
; part 2 uses the real FPU instructions (FLD, FADD, FMUL, FSUB, FSTP).
;
; MASM, x64, Windows console (Visual Studio, ml64).

OPTION CASEMAP:NONE

includelib kernel32.lib

GetStdHandle PROTO
ReadFile     PROTO
WriteFile    PROTO
ExitProcess  PROTO

STD_INPUT_HANDLE  EQU -10
STD_OUTPUT_HANDLE EQU -11

INPUT_BUF_SIZE    EQU 256       ; size of the input buffer

.const

kI1     dd 1
kI2     dd 2
kI3     dd 3
kI5     dd 5
kI6     dd 6
kIm4    dd -4
kI10    dd 10
kI1000  dd 1000
kR2     REAL8 2.0
kR3     REAL8 3.0
kR5     REAL8 5.0
kR6     REAL8 6.0
kRm4    REAL8 -4.0
kR1e6   REAL8 1000000.0

msgTitle     db "Laboratory work No. 4. Variant 14", 13, 10, 13, 10, 0
msgTask      db "Function f(x, y):", 13, 10
             db "    (1 + x) / (5x^2 - y^2 + 1)    if x - y < -4", 13, 10
             db "    3x + 5y^2                     if x - y > 6", 13, 10
             db "    x^3 - 2y                      if -4 <= x - y <= 6", 13, 10
             db "Range: |x| <= 1000, |y| <= 1000. An empty line finishes the
current part.", 13, 10, 0
msgPart1     db 13, 10, "Part 1: integer FPU instructions (x, y are integers,
result is rounded)", 13, 10, 0
msgPart2     db 13, 10, "Part 2: real FPU instructions (x, y are real numbers)",
13, 10, 0
msgX         db "x = ", 0
msgY         db "y = ", 0
msgBranch1   db "Branch 1: x - y < -4, f = (1 + x) / (5x^2 - y^2 + 1)", 13, 10, 0
msgBranch2   db "Branch 2: x - y > 6, f = 3x + 5y^2", 13, 10, 0
msgBranch3   db "Branch 3: -4 <= x - y <= 6, f = x^3 - 2y", 13, 10, 0
msgValue     db "f(x, y) = ", 0
msgCRLF      db 13, 10, 0
msgDivZero   db "Error: division by zero (5x^2 - y^2 + 1 = 0).", 13, 10, 0
msgBadNumber db "Error: this is not a number.", 13, 10, 0
msgRange     db "Error: the value is out of range [-1000; 1000].", 13, 10, 0
msgBye       db 13, 10, "The program is finished.", 13, 10, 0

.code

; ----------------------------------------------------------------------------
; PrintStr - prints a zero-terminated string to the console.
; In:  rcx = address of the string
; ----------------------------------------------------------------------------
PrintStr PROC
    push rbx
    push rsi
    sub  rsp, 38h               ; 20h shadow space + 5th argument + local
```

```
    mov  rsi, rcx
    xor  ebx, ebx               ; ebx = string length
ps_len:
    cmp  byte ptr [rsi + rbx], 0
    je   ps_write
    inc  ebx
    jmp  ps_len

ps_write:
    mov  ecx, STD_OUTPUT_HANDLE
    call GetStdHandle
    mov  rcx, rax               ; hFile
    mov  rdx, rsi               ; lpBuffer
    mov  r8d, ebx               ; nNumberOfBytesToWrite
    lea  r9, [rsp + 28h]        ; lpNumberOfBytesWritten
    mov  qword ptr [rsp + 20h], 0   ; lpOverlapped
    call WriteFile

    add  rsp, 38h
    pop  rsi
    pop  rbx
    ret
PrintStr ENDP

; ------------------------------------------------------------------------
; InputLine - reads a line from the console and removes the end of line.
; In:  rcx = address of the buffer, edx = size of the buffer
; Out: eax = length of the string, or -1 if nothing was read
; ------------------------------------------------------------------------
InputLine PROC
    push rbx
    push rsi
    sub  rsp, 38h

    mov  rsi, rcx               ; rsi = buffer
    mov  ebx, edx
    dec  ebx                    ; ebx = number of bytes to read

    mov  ecx, STD_INPUT_HANDLE
    call GetStdHandle
    mov  rcx, rax               ; hFile
    mov  rdx, rsi               ; lpBuffer
    mov  r8d, ebx               ; nNumberOfBytesToRead
    lea  r9, [rsp + 28h]        ; lpNumberOfBytesRead
    mov  qword ptr [rsp + 20h], 0   ; lpOverlapped
    call ReadFile
    test eax, eax
    jz   il_fail

    mov  ecx, dword ptr [rsp + 28h] ; ecx = number of bytes read
    test ecx, ecx
    jz   il_fail
    xor  eax, eax               ; eax = index in the buffer
il_scan:
    cmp  eax, ecx
    jae  il_end
    mov  dl, byte ptr [rsi + rax]
    cmp  dl, 13                 ; CR
    je   il_end
    cmp  dl, 10                 ; LF
    je   il_end
    inc  eax
    jmp  il_scan

il_end:
    mov  byte ptr [rsi + rax], 0
    jmp  il_exit

il_fail:
    mov  eax, -1
```

```
il_exit:
    add  rsp, 38h
    pop  rsi
    pop  rbx
    ret
InputLine ENDP

; ------------------------------------------------------------------------
; PutUInt - writes an unsigned number as decimal digits.
; In:  rcx = destination, rax = number
; Out: rcx = address after the last digit
; ------------------------------------------------------------------------
PutUInt PROC
    mov  r11, rax               ; r11 = number
    mov  r8d, 1                 ; r8 = number of digits
    mov  r10d, 10
pu_count:
    cmp  rax, 10
    jb   pu_write
    xor  edx, edx
    div  r10
    inc  r8d
    jmp  pu_count
pu_write:
    mov  rax, r11
    lea  r9, [rcx + r8]         ; r9 = address after the last digit
    mov  r11, r9
pu_digit:
    xor  edx, edx
    div  r10
    add  dl, '0'
    dec  r9
    mov  byte ptr [r9], dl
    cmp  r9, rcx
    ja   pu_digit
    mov  rcx, r11
    ret
PutUInt ENDP

; ------------------------------------------------------------------------
; IntToStr - converts a signed 32-bit integer to a zero-terminated string.
; In:  rcx = destination, edx = number
; ------------------------------------------------------------------------
IntToStr PROC
    movsxd rax, edx
    test rax, rax
    jns  is_pos
    mov  byte ptr [rcx], '-'
    inc  rcx
    neg  rax
is_pos:
    call PutUInt
    mov  byte ptr [rcx], 0
    ret
IntToStr ENDP

; ------------------------------------------------------------------------
; RealToStr - converts a REAL8 number to a string with 6 digits after the point.
; In:  rcx = destination, rdx = address of the REAL8 number
; ------------------------------------------------------------------------
RealToStr PROC
    sub  rsp, 28h
    fld  qword ptr [rdx]
    fmul qword ptr [kR1e6]      ; number * 10^6
    frndint                     ; rounded to the nearest integer
    fistp qword ptr [rsp + 20h]
    mov  rax, qword ptr [rsp + 20h]
    test rax, rax
    jns  rs_pos
```

```
    mov  byte ptr [rcx], '-'
    inc  rcx
    neg  rax
rs_pos:
    xor  edx, edx
    mov  r10d, 1000000
    div  r10                    ; rax = integer part, rdx = fraction
    mov  qword ptr [rsp + 20h], rdx
    call PutUInt
    mov  byte ptr [rcx], '.'
    inc  rcx
    mov  rax, qword ptr [rsp + 20h]
    lea  r9, [rcx + 6]
    mov  r11, r9
    mov  r10d, 10
rs_frac:                        ; 6 digits of the fraction with leading zeros
    xor  edx, edx
    div  r10
    add  dl, '0'
    dec  r9
    mov  byte ptr [r9], dl
    cmp  r9, rcx
    ja   rs_frac
    mov  byte ptr [r11], 0
    add  rsp, 28h
    ret
RealToStr ENDP

; ------------------------------------------------------------------------------
; ParseInt - converts a string to an integer in the range [-1000; 1000].
; In:  rcx = string, rdx = address of the DWORD result
; Out: eax = 0 - ok, 1 - not a number, 2 - out of range
; ------------------------------------------------------------------------------
ParseInt PROC
pi_sp1:
    cmp  byte ptr [rcx], ' '
    jne  pi_sign
    inc  rcx
    jmp  pi_sp1
pi_sign:
    xor  r8d, r8d               ; r8d = 1 if the number is negative
    mov  al, byte ptr [rcx]
    cmp  al, '-'
    jne  pi_plus
    mov  r8d, 1
    inc  rcx
    jmp  pi_digits
pi_plus:
    cmp  al, '+'
    jne  pi_digits
    inc  rcx
pi_digits:
    xor  r9d, r9d               ; r9d = value
    xor  r10d, r10d             ; r10d = number of digits
pi_loop:
    movzx eax, byte ptr [rcx]
    sub  eax, '0'
    cmp  eax, 9
    ja   pi_after
    inc  r10d
    cmp  r9d, 1000
    ja   pi_next                ; already out of range, do not grow
    imul r9d, r9d, 10
    add  r9d, eax
pi_next:
    inc  rcx
    jmp  pi_loop
pi_after:
    test r10d, r10d
    jz   pi_bad
```

```
pi_sp2:
    cmp  byte ptr [rcx], ' '
    jne  pi_end
    inc  rcx
    jmp  pi_sp2
pi_end:
    cmp  byte ptr [rcx], 0
    jne  pi_bad
    cmp  r9d, 1000
    ja   pi_range
    test r8d, r8d
    jz   pi_store
    neg  r9d
pi_store:
    mov  dword ptr [rdx], r9d
    xor  eax, eax
    ret
pi_bad:
    mov  eax, 1
    ret
pi_range:
    mov  eax, 2
    ret
ParseInt ENDP

; ------------------------------------------------------------------------
; ParseReal - converts a string "[-]ddd[.ddd]" to a REAL8 number in [-1000;
1000].
; In:  rcx = string, rdx = address of the REAL8 result
; Out: eax = 0 - ok, 1 - not a number, 2 - out of range
; ------------------------------------------------------------------------
ParseReal PROC
    push rbx
    sub  rsp, 30h
    mov  rbx, rdx
pr_sp1:
    cmp  byte ptr [rcx], ' '
    jne  pr_sign
    inc  rcx
    jmp  pr_sp1
pr_sign:
    xor  r8d, r8d               ; r8d = 1 if the number is negative
    mov  al, byte ptr [rcx]
    cmp  al, '-'
    jne  pr_plus
    mov  r8d, 1
    inc  rcx
    jmp  pr_body
pr_plus:
    cmp  al, '+'
    jne  pr_body
    inc  rcx
pr_body:                        ; pass 1: check the syntax
    mov  r9, rcx                ; r9 = start of the number
    xor  r10d, r10d             ; r10d = number of digits
    xor  r11d, r11d             ; r11d = number of digits after the point
    xor  edx, edx               ; edx = 1 after the point
pr_scan:
    movzx eax, byte ptr [rcx]
    cmp  al, '.'
    je   pr_dot
    cmp  al, ','
    je   pr_dot
    sub  eax, '0'
    cmp  eax, 9
    ja   pr_after
    inc  r10d
    add  r11d, edx
    inc  rcx
    jmp  pr_scan
```

```
pr_dot:
    test edx, edx
    jnz  pr_bad                 ; the second point
    mov  edx, 1
    inc  rcx
    jmp  pr_scan
pr_after:
    test r10d, r10d
    jz   pr_bad
pr_sp2:
    cmp  byte ptr [rcx], ' '
    jne  pr_end
    inc  rcx
    jmp  pr_sp2
pr_end:
    cmp  byte ptr [rcx], 0
    jne  pr_bad

    fldz                        ; pass 2: st0 = number without the point
    mov  rcx, r9
    mov  edx, r10d              ; edx = digits left
pr_calc:
    movzx eax, byte ptr [rcx]
    inc  rcx
    cmp  al, '.'
    je   pr_calc
    cmp  al, ','
    je   pr_calc
    sub  eax, '0'
    mov  dword ptr [rsp + 20h], eax
    fimul dword ptr [kI10]      ; st0 = st0 * 10 + digit
    fiadd dword ptr [rsp + 20h]
    dec  edx
    jnz  pr_calc
    mov  ecx, r11d              ; ecx = digits after the point
    test ecx, ecx
    jz   pr_neg
pr_scale:
    fidiv dword ptr [kI10]      ; st0 = st0 / 10
    dec  ecx
    jnz  pr_scale
pr_neg:
    test r8d, r8d
    jz   pr_check
    fchs
pr_check:
    fld  st(0)
    fabs
    fild dword ptr [kI1000]
    fcomip st(0), st(1)         ; compare 1000 with |number|
    fstp st(0)
    jb   pr_range
    fstp qword ptr [rbx]
    xor  eax, eax
    jmp  pr_exit
pr_range:
    fstp st(0)
    mov  eax, 2
    jmp  pr_exit
pr_bad:
    mov  eax, 1
pr_exit:
    add  rsp, 30h
    pop  rbx
    ret
ParseReal ENDP

; ------------------------------------------------------------------------
; ReadInt / ReadReal - print a prompt, read a line and convert it.
; In:  rcx = prompt, rdx = line buffer, r8 = address of the result
```

```
; Out: eax = 0 - ok, 1 - empty line (end of the part), 2 - error (message
printed)
; ------------------------------------------------------------------------
ReadInt PROC
    push rbx
    push rsi
    sub  rsp, 28h
    mov  rbx, rdx
    mov  rsi, r8
    call PrintStr
    mov  rcx, rbx
    mov  edx, INPUT_BUF_SIZE
    call InputLine
    test eax, eax
    jle  ri_end
    mov  rcx, rbx
    mov  rdx, rsi
    call ParseInt
    test eax, eax
    jz   ri_exit
    lea  rcx, msgBadNumber
    cmp  eax, 2
    jne  ri_msg
    lea  rcx, msgRange
ri_msg:
    call PrintStr
    mov  eax, 2
    jmp  ri_exit
ri_end:
    mov  eax, 1
ri_exit:
    add  rsp, 28h
    pop  rsi
    pop  rbx
    ret
ReadInt ENDP

ReadReal PROC
    push rbx
    push rsi
    sub  rsp, 28h
    mov  rbx, rdx
    mov  rsi, r8
    call PrintStr
    mov  rcx, rbx
    mov  edx, INPUT_BUF_SIZE
    call InputLine
    test eax, eax
    jle  rr_end
    mov  rcx, rbx
    mov  rdx, rsi
    call ParseReal
    test eax, eax
    jz   rr_exit
    lea  rcx, msgBadNumber
    cmp  eax, 2
    jne  rr_msg
    lea  rcx, msgRange
rr_msg:
    call PrintStr
    mov  eax, 2
    jmp  rr_exit
rr_end:
    mov  eax, 1
rr_exit:
    add  rsp, 28h
    pop  rsi
    pop  rbx
    ret
ReadReal ENDP
```

```
; ------------------------------------------------------------------------
; PrintBranch - prints the description of the branch.
; In:  ecx = number of the branch (1 - 3)
; ------------------------------------------------------------------------
PrintBranch PROC
    sub  rsp, 28h
    lea  rdx, msgBranch1
    cmp  ecx, 2
    jne  pb_c3
    lea  rdx, msgBranch2
pb_c3:
    cmp  ecx, 3
    jne  pb_print
    lea  rdx, msgBranch3
pb_print:
    mov  rcx, rdx
    call PrintStr
    add  rsp, 28h
    ret
PrintBranch ENDP

; ========================================================================
; Part 1. Integer FPU instructions.
; In:  rcx = address of x (DWORD), rdx = address of y (DWORD),
;      r8 = address of the result (DWORD)
; Out: eax = 0 - ok, 1 - division by zero
; ========================================================================

; f = (1 + x) / (5x^2 - y^2 + 1)
BranchInt1 PROC
    sub  rsp, 28h
    fild  dword ptr [rcx]
    fimul dword ptr [rcx]       ; x^2
    fimul dword ptr [kI5]       ; 5x^2
    fild  dword ptr [rdx]
    fimul dword ptr [rdx]       ; y^2
    fsubp st(1), st(0)          ; 5x^2 - y^2
    fiadd dword ptr [kI1]       ; 5x^2 - y^2 + 1
    fistp dword ptr [rsp + 20h] ; the denominator is stored as an integer
    cmp  dword ptr [rsp + 20h], 0
    je   bi1_zero
    fild  dword ptr [rcx]
    fiadd dword ptr [kI1]       ; 1 + x
    fidiv dword ptr [rsp + 20h] ; (1 + x) / (5x^2 - y^2 + 1)
    fistp dword ptr [r8]
    xor  eax, eax
    jmp  bi1_exit
bi1_zero:
    mov  eax, 1
bi1_exit:
    add  rsp, 28h
    ret
BranchInt1 ENDP

; f = 3x + 5y^2
BranchInt2 PROC
    fild  dword ptr [rcx]
    fimul dword ptr [kI3]       ; 3x
    fild  dword ptr [rdx]
    fimul dword ptr [rdx]       ; y^2
    fimul dword ptr [kI5]       ; 5y^2
    faddp st(1), st(0)
    fistp dword ptr [r8]
    xor  eax, eax
    ret
BranchInt2 ENDP

; f = x^3 - 2y
BranchInt3 PROC
```

```
    fild  dword ptr [rcx]
    fimul dword ptr [rcx]
    fimul dword ptr [rcx]       ; x^3
    fild  dword ptr [rdx]
    fimul dword ptr [kI2]       ; 2y
    fsubp st(1), st(0)
    fistp dword ptr [r8]
    xor  eax, eax
    ret
BranchInt3 ENDP

; CalcInt - chooses the branch by x - y and runs it.
; Out: eax = status of the branch, edx = number of the branch (1 - 3)
CalcInt PROC
    sub  rsp, 28h
    fild  dword ptr [rcx]
    fisub dword ptr [rdx]       ; st0 = x - y
    fild  dword ptr [kIm4]
    fcomip st(0), st(1)         ; compare -4 with x - y
    ja   ci_b1
    fild  dword ptr [kI6]
    fcomip st(0), st(1)         ; compare 6 with x - y
    jb   ci_b2
    fstp st(0)
    call BranchInt3
    mov  edx, 3
    jmp  ci_exit
ci_b1:
    fstp st(0)
    call BranchInt1
    mov  edx, 1
    jmp  ci_exit
ci_b2:
    fstp st(0)
    call BranchInt2
    mov  edx, 2
ci_exit:
    add  rsp, 28h
    ret
CalcInt ENDP

; ============================================================================
; Part 2. Real FPU instructions.
; In:  rcx = address of x (REAL8), rdx = address of y (REAL8),
;      r8 = address of the result (REAL8)
; Out: eax = 0 - ok, 1 - division by zero
; ============================================================================

; f = (1 + x) / (5x^2 - y^2 + 1)
BranchReal1 PROC
    fld1
    fadd qword ptr [rcx]        ; 1 + x
    fld  qword ptr [rcx]
    fmul qword ptr [rcx]        ; x^2
    fmul qword ptr [kR5]        ; 5x^2
    fld  qword ptr [rdx]
    fmul qword ptr [rdx]        ; y^2
    fsubp st(1), st(0)          ; 5x^2 - y^2
    fld1
    faddp st(1), st(0)          ; 5x^2 - y^2 + 1
    fldz
    fcomip st(0), st(1)         ; compare the denominator with 0
    je   br1_zero
    fdivp st(1), st(0)          ; (1 + x) / (5x^2 - y^2 + 1)
    fstp qword ptr [r8]
    xor  eax, eax
    ret
br1_zero:
    fstp st(0)
    fstp st(0)
```

```
    mov  eax, 1
    ret
BranchReal1 ENDP

; f = 3x + 5y^2
BranchReal2 PROC
    fld  qword ptr [rcx]
    fmul qword ptr [kR3]        ; 3x
    fld  qword ptr [rdx]
    fmul qword ptr [rdx]        ; y^2
    fmul qword ptr [kR5]        ; 5y^2
    faddp st(1), st(0)
    fstp qword ptr [r8]
    xor  eax, eax
    ret
BranchReal2 ENDP

; f = x^3 - 2y
BranchReal3 PROC
    fld  qword ptr [rcx]
    fmul qword ptr [rcx]
    fmul qword ptr [rcx]        ; x^3
    fld  qword ptr [rdx]
    fmul qword ptr [kR2]        ; 2y
    fsubp st(1), st(0)
    fstp qword ptr [r8]
    xor  eax, eax
    ret
BranchReal3 ENDP

; CalcReal - chooses the branch by x - y and runs it.
; Out: eax = status of the branch, edx = number of the branch (1 - 3)
CalcReal PROC
    sub  rsp, 28h
    fld  qword ptr [rcx]
    fsub qword ptr [rdx]        ; st0 = x - y
    fld  qword ptr [kRm4]
    fcomip st(0), st(1)         ; compare -4 with x - y
    ja   cr_b1
    fld  qword ptr [kR6]
    fcomip st(0), st(1)         ; compare 6 with x - y
    jb   cr_b2
    fstp st(0)
    call BranchReal3
    mov  edx, 3
    jmp  cr_exit
cr_b1:
    fstp st(0)
    call BranchReal1
    mov  edx, 1
    jmp  cr_exit
cr_b2:
    fstp st(0)
    call BranchReal2
    mov  edx, 2
cr_exit:
    add  rsp, 28h
    ret
CalcReal ENDP

; ------------------------------------------------------------------------
; RunIntPart - input, calculation and output for part 1 (integers).
; ------------------------------------------------------------------------
RunIntPart PROC
    sub  rsp, 158h              ; line buffer [20h], x [120h], y [124h],
                                ; result [128h], status [12Ch], text [130h]
    lea  rcx, msgPart1
    call PrintStr
ip_loop:
    lea  rcx, msgX
```

```
    lea  rdx, [rsp + 20h]
    lea  r8, [rsp + 120h]
    call ReadInt
    cmp  eax, 1
    je   ip_done
    test eax, eax
    jnz  ip_loop
    lea  rcx, msgY
    lea  rdx, [rsp + 20h]
    lea  r8, [rsp + 124h]
    call ReadInt
    cmp  eax, 1
    je   ip_done
    test eax, eax
    jnz  ip_loop

    lea  rcx, [rsp + 120h]
    lea  rdx, [rsp + 124h]
    lea  r8, [rsp + 128h]
    call CalcInt
    mov  dword ptr [rsp + 12Ch], eax
    mov  ecx, edx
    call PrintBranch
    cmp  dword ptr [rsp + 12Ch], 0
    jne  ip_zero

    lea  rcx, [rsp + 130h]
    mov  edx, dword ptr [rsp + 128h]
    call IntToStr
    lea  rcx, msgValue
    call PrintStr
    lea  rcx, [rsp + 130h]
    call PrintStr
    lea  rcx, msgCRLF
    call PrintStr
    jmp  ip_loop
ip_zero:
    lea  rcx, msgDivZero
    call PrintStr
    jmp  ip_loop
ip_done:
    add  rsp, 158h
    ret
RunIntPart ENDP

; ------------------------------------------------------------------------
; RunRealPart - input, calculation and output for part 2 (real numbers).
; ------------------------------------------------------------------------
RunRealPart PROC
    sub  rsp, 168h              ; line buffer [20h], x [120h], y [128h],
                                ; result [130h], status [138h], text [140h]
    lea  rcx, msgPart2
    call PrintStr
rp_loop:
    lea  rcx, msgX
    lea  rdx, [rsp + 20h]
    lea  r8, [rsp + 120h]
    call ReadReal
    cmp  eax, 1
    je   rp_done
    test eax, eax
    jnz  rp_loop
    lea  rcx, msgY
    lea  rdx, [rsp + 20h]
    lea  r8, [rsp + 128h]
    call ReadReal
    cmp  eax, 1
    je   rp_done
    test eax, eax
    jnz  rp_loop
```

```
    lea  rcx, [rsp + 120h]
    lea  rdx, [rsp + 128h]
    lea  r8, [rsp + 130h]
    call CalcReal
    mov  dword ptr [rsp + 138h], eax
    mov  ecx, edx
    call PrintBranch
    cmp  dword ptr [rsp + 138h], 0
    jne  rp_zero

    lea  rcx, [rsp + 140h]
    lea  rdx, [rsp + 130h]
    call RealToStr
    lea  rcx, msgValue
    call PrintStr
    lea  rcx, [rsp + 140h]
    call PrintStr
    lea  rcx, msgCRLF
    call PrintStr
    jmp  rp_loop
rp_zero:
    lea  rcx, msgDivZero
    call PrintStr
    jmp  rp_loop
rp_done:
    add  rsp, 168h
    ret
RunRealPart ENDP

; ------------------------------------------------------------------------
; main - the program entry point.
; All data are local variables on the stack (no global variables).
; ------------------------------------------------------------------------
main PROC
    sub  rsp, 28h

    lea  rcx, msgTitle
    call PrintStr
    lea  rcx, msgTask
    call PrintStr

    call RunIntPart
    call RunRealPart

    lea  rcx, msgBye
    call PrintStr
    xor  ecx, ecx
    call ExitProcess
main ENDP

END
```

Результат:

```
Function f(x, y):
    (1 + x) / (5x^2 - y^2 + 1)    if x - y < -4
    3x + 5y^2                     if x - y > 6
    x^3 - 2y                      if -4 <= x - y <= 6
Range: |x| <= 1000, |y| <= 1000. An empty line finishes the current part.

Part 1: integer FPU instructions (x, y are integers, result is rounded)
x = -3
y = 7
Branch 1: x - y < -4, f = (1 + x) / (5x^2 - y^2 + 1)
f(x, y) = 1
x = 4
y = 9
Branch 1: x - y < -4, f = (1 + x) / (5x^2 - y^2 + 1)
Error: division by zero (5x^2 - y^2 + 1 = 0).
x = 1
y =
Error: this is not a number.
x =
Error: this is not a number.
```

Результат работы раздела 1 и раздела 2

Вывод: Я изучил архитектуру и программную модель сопроцессора; получил практические навыки работы с системами целочисленных и вещественных команд сопроцессора при вычислении значения функции.